

Komplexität von Algorithmen & O-Notation

Probleme, die in Wissenschaft, Technik und Wirtschaft auftreten, erfordern *effiziente Algorithmen*. Dies kann bedeuten, dass Lösungen innerhalb weniger Stunden oder sogar innerhalb von Millisekunden (wie z. B. bei einem Laser-Steuergerät für Augenoperationen) geliefert werden müssen. Um zu beurteilen, ob Programme diesen Anforderungen genügen, muss man die Algorithmen hinsichtlich ihres Bedarfs an Zeit und Speicherplatz analysieren. In diesem Kapitel werden elementare Methoden zur Analyse von Algorithmen vorgestellt. Ein Algorithmus ist umso *effizienter*, je weniger er von den beiden Ressourcen Zeit und Speicher verbraucht.

Zeitaufwand

Anhand unterschiedlicher Algorithmen zur Lösung des gleichen Problems soll gezeigt werden, wie drastisch sich dies – abhängig vom gewählten Algorithmus – auf die Zeit auswirken kann, die zur Lösung benötigt wird.

Die Aufgabenstellung ist hier, eine maximale Abschnittssumme in einem Array von n ganzen Zahlen zu finden. Die maximale Abschnittssumme ergibt sich aus der Teilfolge von aufeinanderfolgenden Zahlen, die unter allen möglichen Teilfolgen die größte Summe liefert. Hat man z. B. die folgende Zahlenreihe:

-59, 52, 46, 14, -50, 58, -87, -77, 34, 15

so liefert die folgende Teil-Zahlenfolge die maximale Abschnittssumme:

52 + 46 + 14 + -50 + 58 = 120

Variante 1

```
int maxfolge1(int z[], int n) {
    int i, j, k, sum, max = -100000000;
    for (i = 0; i < n; i++) {
        for (j = i; j < n; j++) {
            sum = 0;
            for (k = i; k <= j; k++) {
                sum += z[k];
                if (sum > max)
                    max = sum;
            }
        }
    }
    return max;
}
```

$O(n^3)$ $\Rightarrow$ kubisch. Algorithmus

- I Problemgröße
 $\hookrightarrow n$... Arraylänge
- II Kritische Region
- III Best Case | Avg Case | Worst Case
 $\Rightarrow$ alle gleich

Variante 2

```
int maxfolge2(int z[], int n) {
    int i, j, sum, max = -100000000;
    for (i=0; i<n; i++) {
        sum = 0;
        for (j=i; j<n; j++) {
            sum += z[j];
            if (sum > max)
                max = sum;
        }
    }
    return max;
}
```

$n \cdot \frac{n}{2} \Rightarrow O(n^2)$
Quadratisch

- I Problemgröße
 $\hookrightarrow n$... Arraylänge
- II Kritische Region
- III Best Case | Avg Case | Worst Case
 $\Rightarrow$ alle gleich

Vorlesung 3

```
int maxfolge3(int z[], int n) {
    int i, s, gesamtmax = -10000000, endesumme = 0;
    for (i=0; i<n; i++) {
        endesumme = ((s=endesumme+z[i]) > 0) ? s : 0;
        if (endesumme > gesamtmax)
            gesamtmax = endesumme;
    }
    return gesamtmax;
}
```

$O(n)$

Linear-Zusatz

I Problemgröße

↳ n ... Arraylänge

II Kritische Region

III Best Case | Avg Case | Worst Case

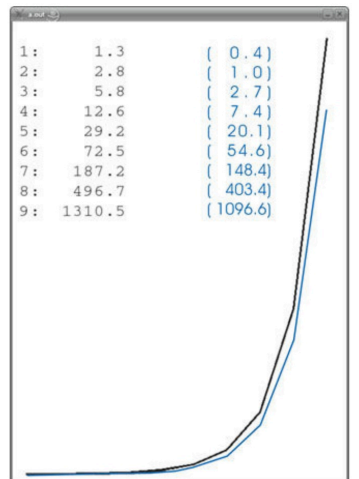
⇒ alle gleich

Weiteres Bsp

Eine natürliche Zahl n ist genau dann eine Primzahl, wenn sie zwischen 2 und $\sqrt{n}$ keine Teiler besitzt. Die folgende Funktion `prim()` testet rekursiv eine beliebige Zahl auf diese Eigenschaft:

```
int prim(int zahl, int teiler) {
    if (zahl < 2 || zahl%2 == 0 || zahl%teiler == 0)
        return 0; /* keine Primzahl */
    else if (teiler*teiler > zahl)
        return 1; /* Primzahl */
    return prim(zahl, teiler+1);
}
```

⇒ exponentiell



Zusammenfassung

Typische Komplexitätsfunktionen zu Algorithmen

1	konstant	Jede Anweisung eines Programms wird höchstens einmal ausgeführt. Dies ist der Idealzustand für einen Algorithmus.
$\log n$	logarithmisch	Speicher- oder Zeitverbrauch wachsen nur mit der Problemgröße n . Die Basis des Logarithmus wird häufig 2 sein, d. h. vierfache Datenmenge verursacht doppelten Ressourcenverbrauch, 8-fache Datenmenge verursacht 3-fachen Verbrauch und 1024-fache Datenmenge 10-fachen Verbrauch.
n	linear	Speicher- oder Zeitverbrauch wachsen direkt proportional mit der Problemgröße n .
$n \log n$	$n \log n$	Der Ressourcenverbrauch liegt zwischen n (linear) und n^2 (quadratisch).
n^2	quadratisch	Speicher- oder Zeitverbrauch wachsen quadratisch mit der Problemgröße. Solche Algorithmen lassen sich praktisch nur für kleine Probleme anwenden.
n^3	kubisch	Speicher- oder Zeitverbrauch wachsen kubisch mit der Problemgröße. Solche Algorithmen lassen sich in der Praxis nur für sehr kleine Problemgrößen anwenden.
2^n	exponentiell	Bei doppelter, dreifacher und 10-facher Datenmenge steigt der Ressourcenverbrauch auf das 4-, 8- bzw. 1024-fache. Solche Algorithmen sind praktisch kaum verwendbar.

↑ verdoppelt

↓

O-Notation

Bezeichnet man die Laufzeit eines Programms (Zeitkomplexität) mit $t(n)$, so ist es offensichtlich, dass mit steigendem n der Koeffizient der höchsten Potenz von n in $t(n)$ zunehmend an Bedeutung für die Laufzeit gewinnt.

Ist $t(n)$ ein Polynom mit n^k ($k > 0$) als höchster Potenz von n , dann hat der zugehörige Algorithmus eine Zeitkomplexität der Größenordnung n^k . Dies wird mit $O(n^k)$ ausgedrückt. Die O-Notation dient dazu, das asymptotische Wachstum einer Funktion abzuschätzen. Sei $t(n)$ die Laufzeit eines Algorithmus, die über die Problemgröße n berechnet wird, dann sagt man „ $t(n)$ ist $O(f(n))$ “ für die Funktionen $t, f: \mathcal{N} \rightarrow \mathcal{N}$, wenn es eine ganze Zahl n_0 und eine Konstante $c > 0$ gibt, dass für alle $n \geq n_0$ gilt: $t(n) \leq c \cdot f(n)$. Die O-Notation ersetzt den Begriff „ist proportional zu“ und ermöglicht das Ignorieren maschinenspezifischer Eigenschaften. Die O-Notation gilt nur für genügend große n und ist unabhängig von den Eigenschaften der Daten und der Implementierung. Sie ist allgemeingültig für alle Ausführungsumgebungen und ist deshalb auch eine Eigenschaft des abstrakten Algorithmus. $O(f(n))$ gibt eine obere Schranke für $t(n)$ an.

Sequenzielle Suche $\rightarrow O(n)$

```
int seqsuche(int z[], int n, int zahl) {
    int i;
    for (i=0; i<n; i++)
        if (z[i] == zahl)
            return i; /* Zahl gefunden */
    return -1; /* Zahl nicht gefunden */
}
```

①

Entscheidungs gefunden

②

ungünstigsten / durchschnittlichen / günstigsten Fall
 Ⓐ Ⓑ Ⓒ

③ streichen oder addieren / multiplizieren Konstanten

Ⓐ Wert nicht vorhanden
 $O(n)$

Ⓑ Durchschnitt
 $n/2$ Schritte $\rightarrow O(n)$

Ⓒ Best-Fall
 ganz vorne 1 Schritt $\rightarrow O(1)$

Schnelles Potenzieren nach Legendre ($O(\log n)$)

$a^b \rightarrow 2^3 \quad 4^7$

```
x = a; y = b; z = 1;
while (y > 0) {
    if (y % 2 == 1)
        z = z * x;
    y = y / 2;
    x = x * x;
}
/* z = Potenz von a hoch b */
```

$$\begin{aligned}
 1 \cdot 4^7 &= 4 \cdot 4^6 = 4(4^2)^3 = 4 \cdot 16^3 \\
 &= 4 \cdot 16 \cdot 16^2 \\
 &= 4 \cdot 16 \cdot 256^1 \\
 &= 16384 \cdot 256^0
 \end{aligned}$$

ungünstig = durchrechnen = günstig

$O(\log(n))$

Primzahlen ($O(n^2)$)

1 2 3 4 5 6 7 8 9 10 11 12 13 14
 ↑ ↑ ↑
 (2, 3, 5 are marked as primes)

```
Eingabe: Zahl n
for (i=1; i<=n; i++) /* prim[0] wird nicht benutzt */
    prim[i] = 1; /* zunächst alle Zahlen erst mal Primzahlen */
for (i=2; i<=n/2; i++)
    if (prim[i]) /* entspricht if (prim[i] != 0) */
        for (j=2*i; j<=n; j=j+i)
            prim[j] = 0;
Ausgabe: Die Primzahlen von 1 bis n sind:
for (i=2; i<=n; ++i)
    if (prim[i])
        Ausgabe: i
```

~~$n + n^2 + n$~~
 $\Rightarrow O(n^2)$

ungünstig = durchrechnen = günstig

Sortieren

11... Module

Bubble Sort

π

```
void bubble_sort(int n, int z[]) {
    for (int i=0; i<n-1; i++)
        for (int j=i+1; j<n; j++)
            if (z[i] > z[j]) {
                int t = z[i]; z[i] = z[j]; z[j] = t; /* Vertauschen von z[i] und z[j] */
            }
}
```

ungünstig = durchschauen = günstig

$$O(n^2)$$

Insert Sort

π

```
void insert_sort(int n, int z[]) {
    for (int i=1; i < n; i++) /* fügt i. te Element ein */
        for (int j=i; j > 0 && z[j] < z[j-1]; j--) {
            int t = z[i]; z[i] = z[j]; z[j] = t; /* Vertauschen von z[i] und z[j] */
        }
}
```

ungünstig: $\frac{n^2}{2} \Rightarrow O(n^2)$

durchschnitt: $\frac{n^2}{4} \Rightarrow O(n^2)$

günstig: $n \Rightarrow O(n)$ (sortiert) π

Select Sort

π

```
void select_sort(int n, int z[]) {
    int i, j, h, t, k;
    for (int i=0; i < n-1; i++) { /* sucht i. tes Element */
        h = i;
        for (int j=i+1; j < n; j++)
            if (z[h] > z[j])
                h = j; /* Merke neue Position */
        if (h != i) {
            int t = z[h]; z[h] = z[i]; z[i] = t; /* Vertauschen von z[h] und z[i] */
        }
    }
}
```

ungünstig = durchschauen = günstig

$$O(n^2)$$

Shell Sort

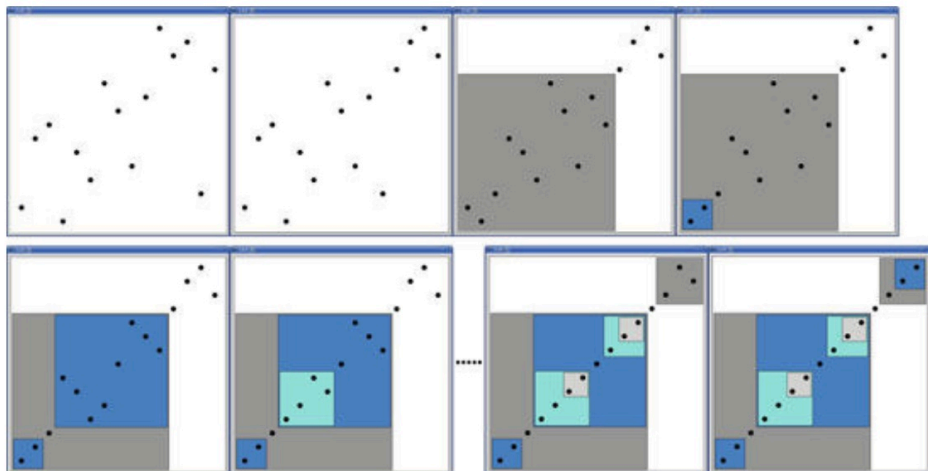
```
void shell_sort(int z[], int l, int r) {
    int h, sw[] = { 1391376, 463792, 198768, 86961, 33936, // Folge von h-Distanzen
                  13776, 4592, 1968, 861, 336, 112, 48, 21, 7, 3, 1 };
    for (int k=0; k < sw.length; k++) {
        h = sw[k];
        for (int i = l+h; i <= r; i++) {
            int v = z[i], j = i;
            while (j >= h && z[j-h] > v) { z[j] = z[j-h]; j -= h; }
            z[j] = v;
        }
    }
}
```

Der Shell-Sort, der von *D.L. Shell* gefunden wurde, war einer der ersten Sortieralgorithmen überhaupt. Er basiert auf dem Insert-Sort, wobei er anders als der Insert-Sort nicht nur benachbarte Elemente, sondern auch weit voneinander liegende Elemente vertauscht. Der Shell-Sort ist eine Folge von sich überlappenden Insert-Sorts, die Elemente in einer Distanz der Größe h voneinander vergleichen und entsprechend sortieren. Man spricht hier auch von *h-sortierten* Daten. Eine *h-sortierte Datenmenge* besteht dabei aus h -unabhängigen sortierten Datenmengen, die übereinander liegen. Dies bedeutet, dass nach jedem h -Durchgang alle Daten, die mit einer Distanz von h zueinander liegen, zueinander sortiert sind. Z. B. könnte man nach einem h -Durchgang mit $h=7$ jedes 7. Element (unabhängig vom Startwert) aus der Datenmenge entnehmen und man hätte ein sortiertes Teilarray.

Quick Sort

```
int partition(int z[], int l, int r) {
    int x = z[r], i = l-1, j = r;
    while (1) {
        while (z[++i] < x)
            ;
        while (z[--j] > x)
            ;
        if (i < j)
            swap(&z[i], &z[j]);
        else {
            swap(&z[i], &z[r]);
            return i;
        }
    }
}

void quick_sort(int z[], int l, int r) {
    if (l < r) {
        int pivot = partition(z, l, r);
        quick_sort(z, l, pivot-1);
        quick_sort(z, pivot+1, r);
    }
}
```



⑧

⑧

[illegible]